

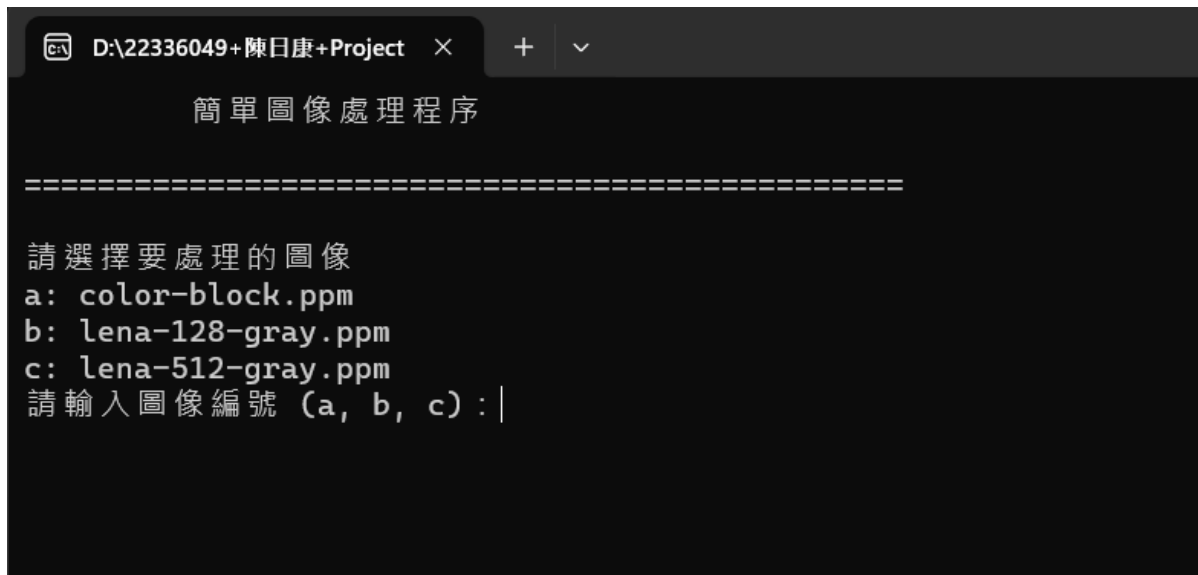
Project 5 实验报告

陳日康 22336049

一、程序功能简要说明

1. 图像加载与显示：可以加载三张图像 (color-block.ppm 、 lena-128-gray.ppm 、 lena-512-gray.ppm) 并显示。
2. 图像处理选项：
 - 压缩并保存：将当前选定的图像进行压缩，并将压缩后的数据保存为二进制文件。
 - 读取解压档并保存：读取之前压缩的二进制文件，解压缩为图像，并保存为新的图像文件而且可选择显示。
 - 转为灰度图像显示并存储：将当前选定的图像转为灰度图像，显示并保存为新的图像文件。
 - 调整图像大小显示并存储：可以对当前选定的图像进行调整尺寸大小，显示并可以保存为新的图像文件。

二、程序运行截图，包括功能演示、部分实际运行结果演示、命令行或交互式界面结果等

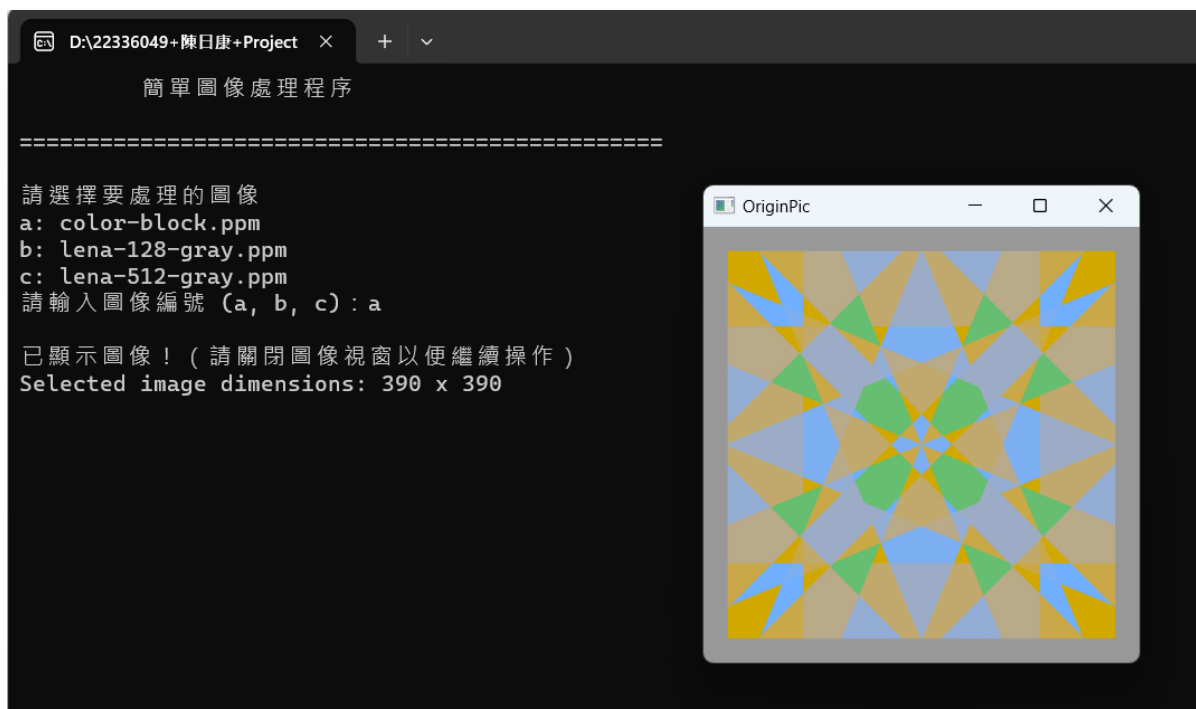


```
D:\22336049+陳日康+Project >
簡單圖像處理程序

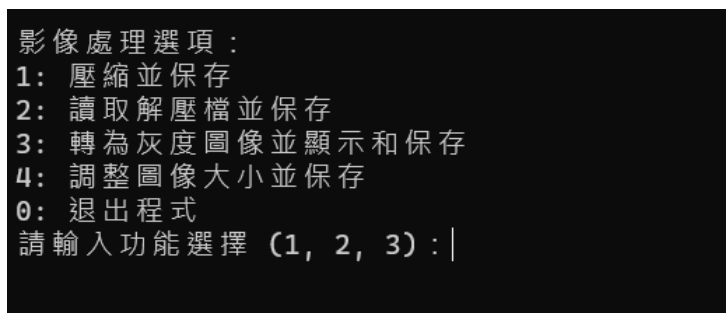
=====

請選擇要處理的圖像
a: color-block.ppm
b: lena-128-gray.ppm
c: lena-512-gray.ppm
請輸入圖像編號 (a, b, c) : |
```

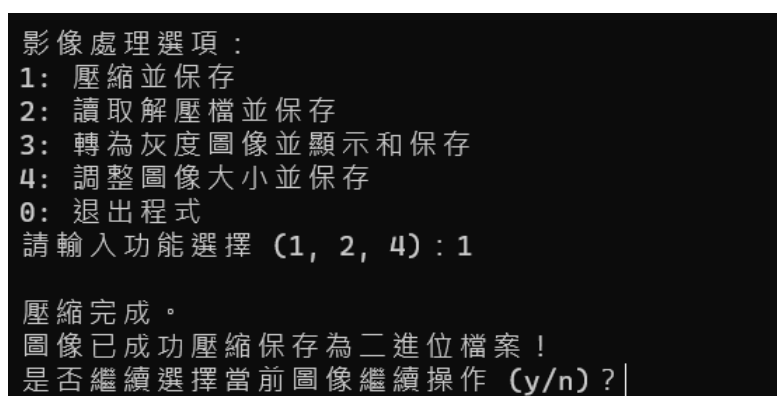
程序一开始会要求用户输入需要处理的图像 (a:color-block.ppm 、 b:len-128-gray.ppm 、 c:len-512-gray.ppm) ，用户可根据自己的需求选择图片。



上图为部分运行结果，当用户完成输入图像选项时，程序会显示用户所选择的图像以及它的尺寸。（用户需关闭图像窗口以便继续操作）



接下来程序会让用户输入需要的操作（1：压缩并保存，2：读取压缩档并保存，3：将彩色图像转换为灰度图像并显示和保存，4：调整图像的大小并保存，0：退出程序）。



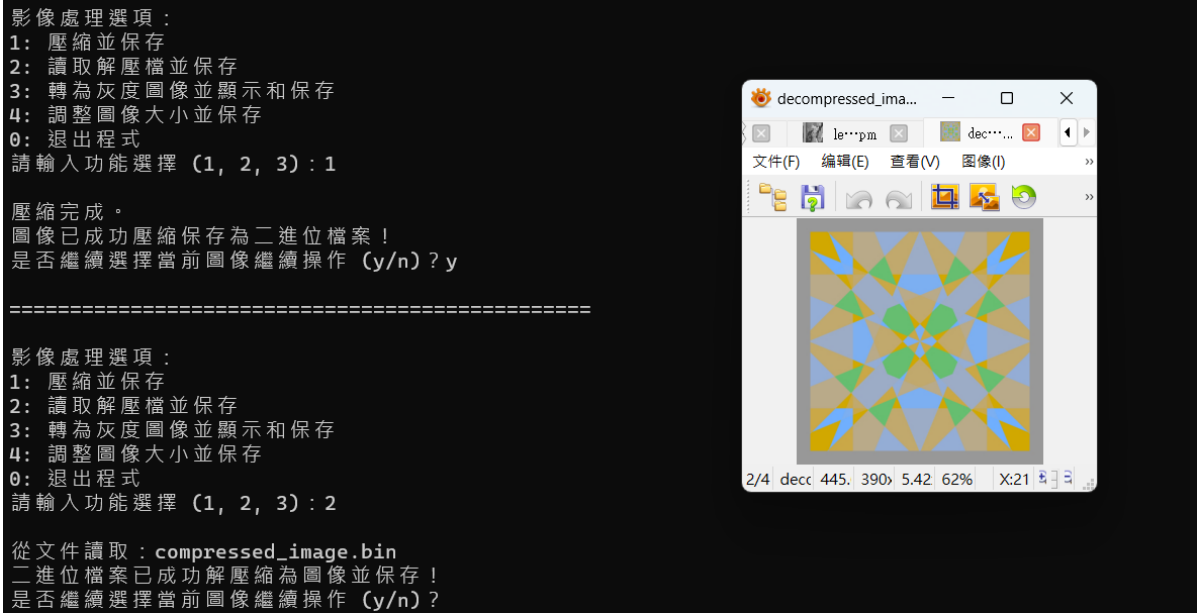
 compressed_image.bin

30/11/2023 23:42

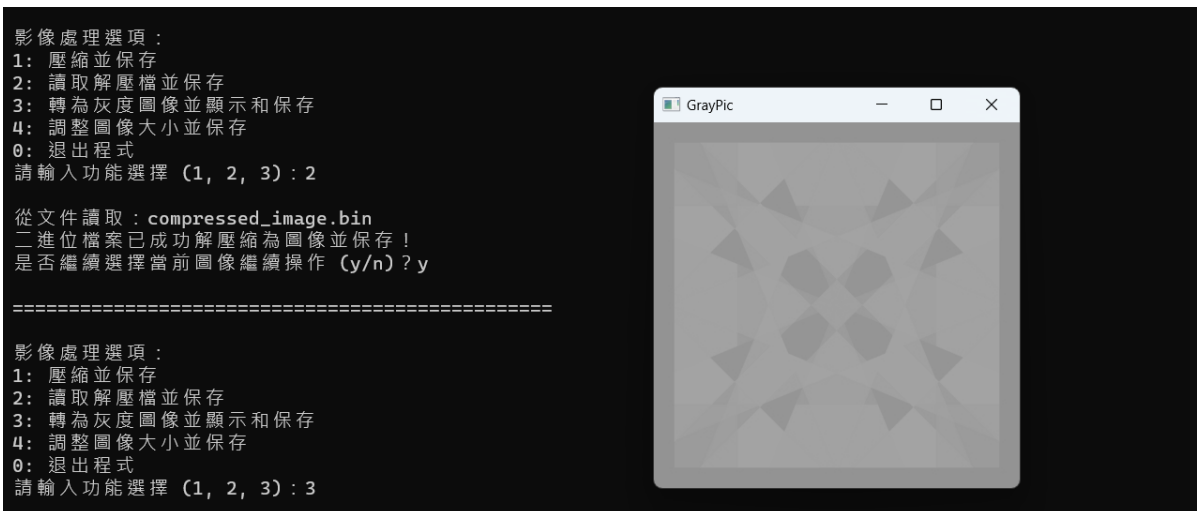
BIN 檔案

2,971 KB

若选择操作1，程序会压缩用户刚才选择的图像并储存在程序所在的路径，而用户需要选择是否对当前图像继续操作（y-是/n-否）。



上图为部分运行结果。若用户选择继续操作当前图像，则程序会返回图像功能选项界面，此时用户可选择操作2或者3，若用户选择操作2，程序会解压缩此前的压缩档，而解压缩档可以成功显示会原图像。



 grayscale-image 30/11/2023 23:55 PPM 檔案 149 KB

上图为部分运行结果。若用户选择图像a（color-block.ppm）和操作3，程序会将color-block.ppm转换为灰度图像并显示和存储（grayscale-image）。

```

影像處理選項：
1: 壓縮並保存
2: 讀取解壓檔並保存
3: 轉為灰度圖像並顯示和保存
4: 調整圖像大小並保存
0: 退出程式
請輸入功能選擇 (1, 2, 3): 3

灰度圖已顯示並保存為新圖像！
圖像已成功保存為 grayscale-image.ppm！

是否繼續選擇當前圖像繼續操作 (y/n)? n

=====

請選擇要處理的圖像
a: color-block.ppm
b: lena-128-gray.ppm
c: lena-512-gray.ppm
請輸入圖像編號 (a, b, c): |

```

```

影像處理選項：
1: 壓縮並保存
2: 讀取解壓檔並保存
3: 轉為灰度圖像並顯示和保存
4: 調整圖像大小並保存
0: 退出程式
請輸入功能選擇 (1, 2, 3): 4

選擇錯誤，請重新選擇！

=====

影像處理選項：
1: 壓縮並保存
2: 讀取解壓檔並保存
3: 轉為灰度圖像並顯示和保存
4: 調整圖像大小並保存
0: 退出程式
請輸入功能選擇 (1, 2, 3): |

```

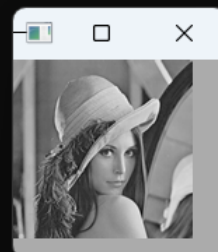
图像a只有3个功能，用户若全部操作完成后，可选n返回图像选择画面选择处理其他图像，若用户输入错误的功能选项，程序会显示错误选择，程序会返回功能选择界面。

```

請選擇要處理的圖像
a: color-block.ppm
b: lena-128-gray.ppm
c: lena-512-gray.ppm
請輸入圖像編號 (a, b, c): b

已顯示圖像！（請關閉圖像視窗以便繼續操作）
Selected image dimensions: 128 x 128

```



在选择图像选择界面，若用户选择图像b，程序会显示图像以及他的尺寸大小。

```
請選擇要處理的圖像
a: color-block.ppm
b: lena-128-gray.ppm
c: lena-512-gray.ppm
請輸入圖像編號 (a, b, c): b
```

```
已顯示圖像！(請關閉圖像視窗以便繼續操作)
Selected image dimensions: 128 x 128
```

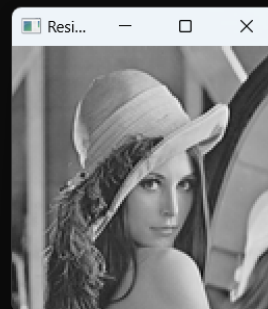
```
=====
```

```
影像處理選項：
```

```
1: 壓縮並保存
2: 讀取解壓檔並保存
3: 轉為灰度圖像並顯示和保存
4: 調整圖像大小並保存
0: 退出程式
```

```
請輸入功能選擇 (1, 2, 4): 4
```

```
請輸入調整後的寬度和高度：256 256
```



resized-image

1/12/2023 0:09

PPM 檔案

65 KB

上图为部分运行结果。若用户输入操作4缩放图像，程序会使用户输入调整后的宽度和高度（测试样例：256 256），输入完成后程序会显示出调整后的图像并存储（resized-image）。而图像c的操作也是一样。

三、部分关键代码及说明

1. 结构定义：表示图像中的像素

```
struct Pixel {
    int x;
    int y;
    int red;
    int green;
    int blue;
};
```

这个结构用于表示图像中的一个像素，包括位置和 RGB 颜色值。

2. 将图像转换为像素结构的向量

```
vector<Pixel> convertToTriplets(const CImg<unsigned char>& image) {
    vector<Pixel> triplets;
    Pixel pixel = {x, y, image(x, y, 0, 0), image(x, y, 0, 1), image(x, y, 0,
2)};
    triplets.push_back(pixel);
}

return triplets;
}
```

convertToTriplets 函数将 CImg 图像对象转换为包含像素结构的向量。遍历图像的每个像素，将其坐标和 RGB 值存储在 Pixel 结构中，并将所有 Pixel 结构存储在一个向量中。

3. 将图像转换为灰度图像

```
CImg<unsigned char> convertToGrayscale(const CImg<unsigned char>& image) {  
    return image.get_RGBtoYCbCr().channel(0);  
}
```

输入参数：CImg 对象表示图像。输出：返回一个灰度图像的 CImg 对象。主要步骤：使用 `get_RGBtoYCbCr` 方法将 RGB 图像转换为 YCbCr 色彩空间。使用 `channel(0)` 仅保留亮度通道。

4. 调整图像大小

```
CImg<unsigned char> resizeImage(const CImg<unsigned char>& image, int  
newwidth, int newHeight) {  
    return image.get_resize(newwidth, newHeight, 1, 1);  
}
```

输入参数：CImg 对象表示图像，新的宽度和高度。输出：返回调整大小后的 CImg 对象。主要步骤：使用 `get_resize` 方法调整图像的大小。

5. 将像素向量压缩并保存到二进制文件

```
void compressAndSave(const CImg<unsigned char>& image, const string& binFileName)  
{  
    // 打開二進位檔案進行寫入  
    ofstream outFile(binFileName, ios::binary);  
  
    if (outFile.is_open()) {  
        // 寫入圖像的寬度和高度  
        const unsigned int width = image.width();  
        const unsigned int height = image.height();  
        outFile.write(reinterpret_cast<const char*>(&width), sizeof(width));  
        outFile.write(reinterpret_cast<const char*>(&height), sizeof(height));  
  
        // 逐個圖元寫入  
        for (int y = 0; y < height; ++y) {  
            for (int x = 0; x < width; ++x) {  
                Pixel pixel = {x, y, image(x, y, 0, 0), image(x, y, 0, 1),  
image(x, y, 0, 2)};  
  
                // 逐個成員變數寫入  
                outFile.write(reinterpret_cast<const char*>(&pixel.x),  
sizeof(pixel.x));  
                outFile.write(reinterpret_cast<const char*>(&pixel.y),  
sizeof(pixel.y));  
                outFile.write(reinterpret_cast<const char*>(&pixel.red),  
sizeof(pixel.red));  
                outFile.write(reinterpret_cast<const char*>(&pixel.green),  
sizeof(pixel.green));  
                outFile.write(reinterpret_cast<const char*>(&pixel.blue),  
sizeof(pixel.blue));  
            }  
        }  
    }  
}
```

```

    }

    cout << "壓縮完成。" << endl;

    // 關閉文件
    outFile.close();
}
else {
    cout << "Error: Could not open file for writing." << endl;
}
}
}

```

遍历图像的每个像素，将其坐标和 RGB 值存储在 `Pixel` 结构中，然后将所有 `Pixel` 结构写入二进制文件。输入参数：`CImg` 对象表示图像，字符串表示输出的二进制文件名。输出：保存二进制文件。主要步骤为：打开二进制文件进行写入，写入图像的宽度和高度，使用双重循环遍历图像的每个像素，将像素信息写入二进制文件。

6. 从二进制文件读取并解压缩成图像

```

void readAndDecompress(const string& binFileName, const string&
outputImageName) {
    cout << "從文件讀取：" << binFileName << endl;
    // 打開二進位檔案進行讀取
    ifstream inFile(binFileName, ios::binary);

    if (!inFile) {
        cerr << "Error: Could not open file for reading: " << binFileName <<
endl;
        return;
    }

    // 讀取圖像的寬度和高度
    unsigned int width, height;
    inFile.read(reinterpret_cast<char*>(&width), sizeof(width));
    inFile.read(reinterpret_cast<char*>(&height), sizeof(height));

    // 創建用於解壓縮的圖像
    CImg<unsigned char> decompressedImage(width, height, 1, 3, 0);

    // 逐個圖元讀取
    for (unsigned int y = 0; y < height; ++y) {
        for (unsigned int x = 0; x < width; ++x) {
            Pixel pixel;

            // 逐個成員變數讀取
            inFile.read(reinterpret_cast<char*>(&pixel.x), sizeof(pixel.x));
            inFile.read(reinterpret_cast<char*>(&pixel.y), sizeof(pixel.y));
            inFile.read(reinterpret_cast<char*>(&pixel.red),
sizeof(pixel.red));
            inFile.read(reinterpret_cast<char*>(&pixel.green),
sizeof(pixel.green));
            inFile.read(reinterpret_cast<char*>(&pixel.blue),
sizeof(pixel.blue));

```

```

        // 填充解壓後的圖像
        decompressedImage(pixel.x, pixel.y, 0, 0) = pixel.red;
        decompressedImage(pixel.x, pixel.y, 0, 1) = pixel.green;
        decompressedImage(pixel.x, pixel.y, 0, 2) = pixel.blue;
    }
}

// 保存解壓後的圖像
decompressedImage.save(outputImageName.c_str());

// 關閉文件
inFile.close();
}

```

输入参数：字符串表示输入的二进制文件名，字符串表示输出的图像文件名。输出：保存解压缩后的图像。主要步骤：打开二进制文件进行读取。读取图像的宽度和高度。使用双重循环读取每个像素的信息，将其还原为图像。保存解压缩后的图像。

7. 显示灰度图像

```

void displayGrayscale(const CImg<unsigned char>& originalImage) {
    CImg<unsigned char> grayscaleImage = convertToGrayscale(originalImage);

    CImgDisplay grayscaleDisplay(grayscaleImage, "GrayPic");
    while (!grayscaleDisplay.is_closed()) {
        grayscaleDisplay.wait();
    }
    cout << "灰度圖已顯示並保存為新圖像！" << endl;
    string outputFileName = "grayscale-image.ppm";
    grayscaleImage.save(outputFileName.c_str());
    cout << "圖像已成功保存為 " << outputFileName << "！" << endl;
    cout << endl;
}

```

输入参数：CImg 对象表示图像。输出：显示灰度图像。主要步骤为：将输入图像转换为灰度图像。使用 `CImgDisplay` 函数显示灰度图像。

8. 调整图像大小并显示

```

void resizeAndDisplay(const CImg<unsigned char>& originalImage, int newWidth,
int newHeight) {
    CImg<unsigned char> resizedImage = resizeImage(originalImage, newWidth,
newHeight);

    CImgDisplay resizedDisplay(resizedImage, "ResizedPic");
    while (!resizedDisplay.is_closed()) {
        resizedDisplay.wait();
    }

    cout << "圖片已調整大小並保存為新圖像！" << endl;
    string outputFileName = "resized-image.ppm";
}

```

```
resizedImage.save(outputFileName.c_str());  
cout << "圖片已成功保存為 " << outputFileName << "!" << endl;  
cout << endl;  
}
```

輸入參數：CImg 對象表示圖像，新的寬度和高度。輸出：顯示調整大小後的圖像。使用 CImgDisplay 顯示調整大小後的圖像。

四、程序運行方式簡要說明

1. 選擇圖像：程序運行後，用戶將看到一個圖像選擇菜單。輸入相應的字母 (a, b, 或 c) 選擇圖像。比如，輸入 **a** 選擇 **color-block.ppm**。
2. 顯示圖像：選擇圖像後，程序將顯示選定的圖像。關閉圖像窗口以繼續。
3. 選擇圖像處理操作：在程序主菜單中，輸入數字以選擇不同的圖像處理操作：
 - 輸入 **1** 進行圖像壓縮並保存。
 - 輸入 **2** 讀取並解壓保存的二進制文件。
 - 輸入 **3** 將圖像轉為灰度圖像並顯示。
 - 輸入 **4** 調整圖像大小並保存。
 - 輸入 **0** 退出程序。
4. 交互操作：根據選擇，程序會進行相應的圖像處理操作，並顯示處理後的圖像。在執行完一項操作後，你將看到程序的提示信息。
5. 選擇返回或退出：在進行圖像處理操作後，程序會詢問是否繼續對當前圖像進行操作。輸入 **y** 表示繼續對當前圖像進行操作，輸入 **n** 表示返回圖像選擇菜單。在圖像選擇菜單中，輸入 **0** 退出程序。
6. 程序結束：輸入 **0** 退出程序後，程序結束運行。